

FarmFresh Java — Project Plan

College group project · Core Java + MongoDB · Ported from the FarmFresh MERN marketplace concept

1. Package Structure

```
com.farmfresh
├── model/
│   ├── User.java           (abstract base class)
│   ├── Farmer.java        (extends User)
│   ├── Consumer.java      (extends User)
│   ├── Product.java
│   └── Order.java
├── repository/
│   ├── FarmerRepository.java
│   ├── ConsumerRepository.java
│   ├── ProductRepository.java
│   └── OrderRepository.java
├── service/
│   ├── ProductService.java
│   ├── OrderService.java
│   └── TrendingService.java
├── exception/
│   ├── OutOfStockException.java
│   └── ProductNotFoundException.java
├── util/
│   └── MongoConnection.java
└── Main.java
```

2. Classes — Fields & Responsibilities

User (abstract)

```
abstract class User {
    String id;           // Mongo ObjectId as String
    String name;
    String email;
    String password;
}
```

Base class for shared identity fields. Forces Farmer/Consumer to implement any abstract methods (e.g. `displayMenu()`).

Farmer extends User

```
class Farmer extends User {
    List<String> productIds; // products this farmer lists
}
```

Behavior: add product, update stock, remove product, view own listings.

Consumer extends User

```
class Consumer extends User {
    List<String> orderIds; // orders placed
}
```

Behavior: browse products, place order, view order history.

Product

```
class Product {
    String id;
    String name;
    String farmerId;
    double price;
    int quantity;
    int orderCount;    // incremented each time it's ordered - powers trending
}
```

Order

```
class Order {
    String id;
    String consumerId;
    String productId;
    int quantityOrdered;
    Date timestamp;
    String status;    // "PLACED", "COMPLETED", "CANCELLED"
}
```

3. Relationships

- Farmer 1 → many Product (via farmerId field on Product, referenced not embedded — mirrors the MERN referencing style)
- Consumer 1 → many Order
- Product 1 → many Order (a product can be ordered many times)
- Inheritance: Farmer and Consumer both extend User — shared login/identity logic lives once in the parent
- No multiple inheritance needed — for shared behavior across unrelated classes later (e.g. a Notifiable interface), use interface, not extension

4. Repository Layer (Mongo DAO)

Each repository wraps one collection and exposes only clean methods — no raw Mongo code leaks into service/Main.

```
class ProductRepository {
    MongoClient<Document> collection;

    void insert(Product p) {...}
    Product findById(String id) {...}
    List<Product> findAll() {...}
    List<Product> findByFarmerId(String farmerId) {...}
    void updateQuantity(String id, int newQty) {...}
    void incrementOrderCount(String id) {...}
    List<Product> findTrending(int limit) {...} // aggregation, sort by orderCount desc
    void delete(String id) {...}
}
```

Same pattern applies to FarmerRepository, ConsumerRepository, OrderRepository.

MongoConnection (singleton)

```
class MongoConnection {
    private static MongoClient client;
    private static MongoDBDatabase db;
    static MongoDBDatabase getDatabase() {
        if (client == null) {
            client = MongoClient.create("mongodb://localhost:27017");
            db = client.getDatabase("farmfresh");
        }
        return db;
    }
}
```

5. Service Layer (business logic — no Mongo code here)

ProductService — addProduct(...), updateStock(...), removeProduct(...)

OrderService — placeOrder(consumerId, productId, qty):

- 1 Fetch product via ProductRepository
- 2 Check quantity >= qty → else throw OutOfStockException
- 3 Decrement stock, increment orderCount
- 4 Create Order, save via OrderRepository
- 5 Append order id to consumer's orderIds

TrendingService — getTrendingProducts(limit) → calls ProductRepository.findTrending()

6. Exceptions

```
class OutOfStockException extends RuntimeException { ... }
class ProductNotFoundException extends RuntimeException { ... }
```

Caught in Main around user actions, printing friendly console messages.

7. Main.java — Console Flow

1. Login/Register as Farmer or Consumer
2. If Farmer -> Add/Update/Remove product, view my listings
3. If Consumer -> Browse products, view trending, place order, view my orders
4. Exit

Simple Scanner-driven menu loop with a switch statement.

8. Recommended Build Order

- 1 MongoConnection + verify DB connectivity
- 2 Model classes (User, Farmer, Consumer, Product, Order)
- 3 Repositories (CRUD first, aggregation/trending last)
- 4 Exceptions
- 5 Services (wire repositories together)

6 Main console menu last — it's just gluing everything together

9. Team Split

Person	Owns
A	Models + MongoConnection + Maven/Gradle setup
B	FarmerRepository + ProductRepository + ProductService
C	ConsumerRepository + OrderRepository + OrderService (incl. OutOfStockException logic)
D	Main console menu + TrendingService + exceptions + testing/demo prep

10. Project Folder vs GitHub Repo — Clarified

- **Locally / in your IDE:** one project folder (e.g. FarmFreshJava/) containing subfolders like model/, repository/, service/, each holding multiple .java files. Not one giant file.
- **On GitHub:** one repository = one project = the entire folder structure pushed as-is (src/, pom.xml or build.gradle, README.md). Same mental model as your FarmFresh MERN repo which had routes/, models/, controllers/ folders in one repo.
- **.gitignore:** exclude target/ or build/ (compiled output) and any .env/Mongo credentials before pushing.
- **README.md:** worth writing — explains structure and how to run the project for your teacher and teammates.
- **Team workflow:** use branches per person/module (e.g. farmer-module, order-module) and merge via pull requests rather than everyone pushing directly to main.

Next step: write MongoConnection.java and the model classes.